



Programación de Estructuras de Datos y Algoritmos Fundamentales

Actividad Integradora 1 Conceptos básicos y algoritmos fundamentales

Farid Gabriel Velasco Martínez A01736669

24 de marzo de 2025

Índice

Índice	2
Introducción	3
Análisis del algoritmo de ordenamiento	4
Análisis del algoritmo de búsqueda 1	5
Análisis del algoritmo de búsqueda 2	6
Conclusión	7

Introducción

La situación problema plantea los peligros de una botnet, la cual es una red de dispositivos y sistemas digitales infectados con software malicioso que ya sea automáticamente o a discreción, ejecutan software malicioso dentro del dispositivo infectado, infectan a otros dispositivos, lanzan ataques DDOS, phishing, spamming, ejecutan spyware, filtran datos, o realizan algún otro sin fin de actividades maliciosas y peligrosas a las que dispositivos digitales conectados a internet y mal protegidos o con protocolos de red desactualizados puedan ser vulnerables.

En esta primera actividad integradora se nos plantea una situación en la que una red genera registros de intentos de ataque en un archivo de texto, nuestra tarea es desarrollar una aplicación que sea capaz de ordenar los registros por fecha y permita realizar 2 tipos de búsqueda, una búsqueda directa de un registro y la búsqueda del primer par de registros con un número específico de días entre ellos.

Existen diversos algoritmos de ordenamiento y búsqueda, sin embargo para una situación como la anteriormente mencionada, es de suma importancia contar con aquellos que realicen la tarea deseada de la manera más óptima posible, ya que el procesamiento y análisis de datos eficiente es crucial para prevenir o poder responder a tiempo a ataques maliciosos en una red, y de este modo asegurar la protección de los datos y los equipos que la componen. Es incluso posible que ciertos algoritmos sean más eficientes en ciertos escenarios que otros, por ejemplo, el algoritmo TimSort utiliza Insertion Sort para pequeños bloques de datos (menores a 32 elementos) en su proceso de ordenamiento[4] ya que este a pesar de tener una complejidad temporal de $O(n^2)$ en este caso puede comportarse como $O(n)$ siendo más rápido y menos costoso que utilizar otra iteración de Merge Sort o Quick Sort cuyas complejidades computacionales y temporales los hacen menos adecuados, por lo que comprender el funcionamiento de cada uno es útil para optimizar lo más posible la aplicación.

Análisis del algoritmo de ordenamiento

Nuestro algoritmo de ordenamiento que denominamos 'Two Step Sort' consta de la combinación de los algoritmos Counting Sort, que es un algoritmo de ordenamiento que no se basa en comparaciones, si no en el conteo de las apariciones de cada valor posible en un vector, y Quick Sort para reducir la complejidad y tiempo de ordenamiento de los registros:

En primer lugar aprovechamos que el objeto 'Registro' cuenta con varios atributos numéricos que lo representan, en este caso los de interés son:

- Día del año
- Segundos totales de la hora de cada Registro

Utilizamos Counting Sort con el atributo 'y_day' para ordenar el vector agrupando por día del año. Counting sort tiene una complejidad $O(n + k)$ por tanto:

$$n = 16809 \quad k = 365 \quad O(16809 + 365) \approx O(n)$$

Durante Counting Sort agregamos pasos intermedios; validamos las posiciones donde se encuentran días válidos recorriendo k posiciones en el vector de conteo, después obtenemos los valores en esas posiciones recorriendo de nuevo el vector de conteo k posiciones, lo que incrementa la complejidad a $O(n + k + k + k)$ que sigue siendo despreciable frente a n . Con el paso anterior obtenemos los límites donde se encuentra el último dato de un subconjunto de días, el cual está en su posición correspondiente por día pero no por hora.

Posteriormente aplicamos Quick Sort entre cada límite con el atributo 't_seg' para ordenar el vector por segundos totales de la hora de cada día tal que:

Quick Sort de 0 a bounds [0]

Quick Sort de bounds [0] a bounds [1]

Quick Sort de bounds [i] a bounds [i+1]

Así hasta -> Quick Sort de bounds [j] a bounds[j+1] (j es el penúltimo elemento)

Y termina el algoritmo.

Al aplicar Quick Sort sobre subconjuntos del vector que ya están ordenados por día, se reduce significativamente la profundidad del árbol de recursión y mejora la eficiencia. El peor caso teórico con este enfoque sería $O(n \log n)$, sin embargo en el caso más óptimo:

Existen d días distintos, y cada uno tiene $\frac{n}{d}$ elementos aproximadamente, entonces para cada subconjunto tenemos $O(\frac{n}{d} \log \frac{n}{d})$ y al haber d subconjuntos:

$$O(d \cdot \frac{n}{d} \log \frac{n}{d}) = O(n \log \frac{n}{d})$$

Entonces la complejidad total del algoritmo sería $O(n + k \cdot 3 + n \log \frac{n}{d}) = O(n \log \frac{n}{d})$

Si computamos la complejidad teórica y la comparamos con el resultado experimental del total de comparaciones realizadas obtenemos:

$$16809 + 365 \cdot 3 + 16809 \cdot \log \frac{16809}{365} = 110777 \approx 113741$$

Lo cual es menos de la mitad de las comparaciones realizadas al ordenar el vector con Quick Sort puro: 286667 y en distribuciones más normalizadas Two Step Sort puede incluso llegar a las 92873 comparaciones teóricas. Esto reduce de gran manera el tiempo de ejecución del programa cumpliendo con el objetivo de procesar la información de la manera más eficiente posible.

Análisis del algoritmo de búsqueda 1

Se nos pide implementar un algoritmo de búsqueda que sea eficiente para realizar un número B de operaciones. Actualmente contamos con un vector ordenado, por lo que la elección del algoritmo de búsqueda de registros fue sencilla, el algoritmo más eficiente para ello es Binary Search.

Exploramos otras posibilidades como ordenar las fechas solicitadas de búsqueda y luego comparar sincrónicamente cada vector, sin embargo el paso previo de ordenar las solicitudes de búsqueda consume recursos y tiempo que no se compensan, por lo que repetir Binary Search B veces es el mejor enfoque:

$$O(B \cdot \log n)$$

Que en términos prácticos y en el mejor caso (a menos que $B \geq n$), la complejidad seguirá siendo $O(\log n)$. Esto nos da la rapidez de búsqueda deseada y optimiza nuestra aplicación.

Análisis del algoritmo de búsqueda 2

Por último, se requiere hacer la búsqueda del primer par de registros con D días de diferencia entre ellos. Para esto se utilizó la técnica ‘Two Pointers’ para poder realizar la búsqueda en tiempo lineal $O(n)$.

Nuestro algoritmo ‘find_by_interval’ recibe un vector ‘vals’ ordenado, la diferencia entre valores a buscar ‘D’ y por cuestiones de diseño de la clase un entero ‘int element’ que es el índice del elemento en el array de atributos de tipo int a partir del cual realizar la búsqueda.

Pseudocódigo:

```
function find_by_interval(vals, D, int_element) {
    Crear un arreglo pair con 2 elementos inicializados en -1
    l ← 0
    r ← 1
    while l ≤ r Y r < tamaño de vals do {
        diff ← vals[r](int_element) - vals[l](int_element)
        if diff = D Y l ≠ r {
            pair[0] ← l
            pair[1] ← r
            Retornar pair
        }
        if diff > D entonces l ← l + 1
        Sino r ← r + 1

    Retornar pair
}
```

Básicamente el algoritmo va leyendo los valores con dos punteros, 'l' y 'r'. 'r' está adelantado para compensar si es que la diferencia entre l y r sea menor a la deseada, en este caso r avanza, si no, para reducir la diferencia l avanza. El peor caso ocurre cuando no hay ningún par con diferencia 'D', por lo que el algoritmo recorre el vector 2 veces (1 vez por cada puntero): $O(2n) \approx O(n)$ por lo que sigue siendo bastante eficiente.

La técnica de two pointers es una de las más óptimas para este problema por ello la implementamos con el fin de salvar carga computacional y tiempo de ejecución lo más posible que a comparación de otros algoritmos usuales de búsqueda por pares, no tiene una gran carga computacional.

Conclusión

Durante la resolución de esta primera actividad integradora me percaté de la importancia crucial que tienen los algoritmos de búsqueda y ordenamiento eficientes. En una situación real donde un sistema de red esté sufriendo ataques, detectar las amenazas a tiempo garantiza la seguridad tanto del sistema como de todos los usuarios válidos que la usan y los dispositivos que la componen, la protección de la información sensible etc.

Actualmente los dispositivos más vulnerables al tipo de ataques como los vistos son los dispositivos IoT y dispositivos conectados a internet con protecciones desactualizadas y firewalls obsoletos o ineficientes[10], por lo que una solución que ataque al problema casi de raíz sería que las personas mantengan sus dispositivos protegidos y que las empresas hagan más robustos los protocolos de comunicación y de protección de sus productos que se conectan a la red, sin embargo no todos tienen los conocimientos o la infraestructura necesaria para hacerlo, por lo que queda a manos de los programadores y expertos en ciberseguridad implementar medidas de seguridad rápidas, confiables e innovadoras para frenar los actos maliciosos e ilícitos en el mundo digital.

Referencias:

1. *Acelera tus operaciones con IOT.* (s. f.).
<https://www.oracle.com/mx/internet-of-things/>
2. Condliffe, J. (2020, 2 abril). The FBI Shut Down a Huge Botnet, but There Are Plenty More Left. *MIT Technology Review*.
<https://www.technologyreview.com/2017/04/11/152645/the-fbi-shut-down-a-huge-bot-net-but-there-are-plenty-more-left/>
3. GeeksforGeeks. (2023a, septiembre 22). *Date and Time Parsing in C++*.
GeeksforGeeks. <https://www.geeksforgeeks.org/date-and-time-parsing-in-cpp/>
4. GeeksforGeeks. (2023b, noviembre 20). *TimSort Data Structures and Algorithms Tutorials*. GeeksforGeeks. <https://www.geeksforgeeks.org/timsort/>
5. GeeksforGeeks. (2024, 20 diciembre). *Find a pair with the given difference*.
GeeksforGeeks. <https://www.geeksforgeeks.org/find-a-pair-with-the-given-difference/>
6. GeeksforGeeks. (2025a, enero 30). *Counting Sort Data Structures and Algorithms Tutorials*. GeeksforGeeks. <https://www.geeksforgeeks.org/counting-sort/>
7. GeeksforGeeks. (2025b, febrero 4). *Quick sort*. GeeksforGeeks.
<https://www.geeksforgeeks.org/quick-sort-algorithm/>
8. GeeksforGeeks. (2025c, febrero 19). *Binary Search Algorithm Iterative and Recursive Implementation*. GeeksforGeeks. <https://www.geeksforgeeks.org/binary-search/>
9. GeeksforGeeks. (2025d, marzo 22). *Insertion Sort Algorithm*. GeeksforGeeks.
<https://www.geeksforgeeks.org/insertion-sort-algorithm/>
10. Robinson, S., Hanna, K. T., & Lutkevich, B. (2025, 18 febrero). *What is a botnet?*
Search Security.
https://www.techtarget.com/searchsecurity/definition/botnet?track=NL-1823&ad=931942&src=931942&asrc=EM_NLN_122529935